

Technical Brief — Serio Ludere Preview Catalog & Admin

For: Astro Developer **Owner:** Ramez Al-Syoufi / Eclipse Design **Version:** 0.5 — reconciled against the `preview` Figma file

The Figma file is now the source of truth for UI, tokens, IA and motion. Page `07` · `Handoff` carries token provenance, responsive intent, the motion spec, the component inventory, the QA audit, and the decision log. Read it before building. This brief is the source of truth for data, auth, scraping and storage.

1. Scope

An Astro app on `preview.serioludere.com` with two independent password gates:

- `/admin` — internal tool: scrape one supplier product at a time, review it, commit it to a Google Sheet (data) and Google Drive (images)
- `/{{customer-slug}}` — private catalog preview for one named buyer, recording visits and likes/dislikes

There is no separate database. One Google Spreadsheet is the entire datastore. Shopify is not the host.

2. Storage model

One spreadsheet. Products in a single tab; other entities each get their own tab in the same file.

Tab	Shape	Written by
<code>Products</code>	One row per product, Shopify-CSV column set	Admin only
<code>Collections</code>	One row per collection	Admin only
<code>Customers</code>	One row per customer	Admin only

Reactions	Append-only event log	Customers, concurrently
Visits	Append-only event log	Customers, concurrently

Images live in Google Drive (§9). Session state, rate-limit counters, and the buyer's unit/currency choice live in memory and cookies — **never** in the sheet.

3. Making Sheets work as a database

Sheets has no locking and no transactions. Product data is single-writer and fine. Reactions and visits are written by multiple buyers at once, and a naive read-modify-write will silently lose updates. Three rules make this safe. **Treat them as requirements, not advice.**

Rule 1 — Reactions and Visits are append-only, never updated

```
Reactions:  event_id, customer_slug, product_id, reaction, source, created_at
Visits:     event_id, customer_slug, occurred_at, user_agent, referrer
```

`reaction ∈ like | dislike | none`. `source ∈ card | detail` (the design allows liking from the grid but only disliking from the detail page — see §7). Current state is **derived**: latest event per `(customer_slug, product_id)` wins. Un-liking appends a `none` event; it never edits a row.

`values.append` does not read before writing, so two simultaneous writers cannot clobber each other. No in-process queue fixes this on serverless — each request may land on a different cold instance.

Rule 2 — Batch writes on the client

Quota is 60 write requests/min/user. Buffer reactions client-side for 2–3 seconds and flush as **one append call with multiple rows**. Flush on `visibilitychange` and page hide so nothing is lost on tab close.

Rule 3 — Cache reads, compact periodically

Cache every tab in memory, 60s TTL, manual refresh in the admin. Derive reaction state once per cache cycle. Add a **compact reactions** maintenance action that rewrites the tab keeping only the latest event per pair — manual, never automatic, never while a buyer is active.

4. Stack

Concern	Choice	Note
Framework	Astro, <code>output: 'server'</code>	Must be SSR
Adapter	Vercel or Netlify, Node runtime	Not Edge — <code>googleapis</code> unsupported, uploads exceed CPU limits
Data	Google Sheets API v4	Everything. No DB.
Images	Google Drive API v3	See §9
UI islands	React or Svelte	Drawer, fetch modal, tables, like/dislike, unit toggle, currency picker
Styling	Tailwind + CSS variables generated from the Figma variables	See §5
DNS	CNAME <code>preview</code> → host	Confirm registrar

5. Design system — implement from the Figma variables

The token set is **extracted from the live Prestige theme**, not invented. 07 · Handoff gives every primitive's source (a `settings_data` key or a CSS custom property) or marks it `NO SOURCE`. Do not re-derive values from screenshots.

Three semantic modes

Preview · Admin · Admin Dark. Ship as three CSS custom-property sets on `:root / [data-mode]`. Every component reads semantics only — never a primitive, never a raw value.

Values you will hit immediately

Type	Inter for body and headings (<code>text_font = inter_n4</code> , <code>heading_font = inter_n7</code>). Body base 13px , scale 11/12/13/15/19. Headings 28.8 / 25.2 / 19.8 / 16.2 / 12.6 / 11.2 (Prestige's fluid clamps × the 0.9 <code>heading_font_size_factor</code>)
	0 on buttons, inputs, cards. <code>radius/pill</code> (9999) only for chips and the

Radius	reaction circles
Borders	1px, the only stroke weight the theme uses
Focus ring	Ink at 2px — deliberately not the brand red, because red is the error colour and a red ring on a red-bordered invalid field is unreadable
Canvas	Preview <code>#FEFCF0</code> · Admin <code>#F8F7F5</code> · Admin Dark <code>#141413</code>
Surface	Preview <code>#FFFFFF5</code> (the paper the storefront puts behind every rug image)
Brand red	<code>#B80D09</code> — primary hover and destructive actions
Feedback	success <code>#307A07</code> · warning <code>#ED8A00</code> · danger <code>#CB2B2B</code>
Containers	440 / 680 / 980 / 1150 / 1260 / 1360
Mono	IBM Plex Mono — no theme source; required for Product IDs and copyable ID lists

The asymmetry is one alias

`space/section` resolves to **112** in Preview and **24** in Admin. `motion/transition` resolves to **600ms** in Preview and **150ms** in Admin. Those two aliases are what make the tool dense and the catalog editorial. Do not hardcode around them.

Off-grid values are intentional

`input-y 10.4` · `input-x 12.8` · `control-gap 10` · `checkbox 14` · `container/md 1150` .

These are Prestige's real values, carried rather than rounded. Rounding them breaks the visual match with the storefront. The `inset/*` namespace (44 / 34 / 10 / 20) is iOS chrome, not spacing.

Three theme defects worth knowing

- `--shadow-block` interpolates settings that do not exist in `settings_schema.json` and emits invalid CSS. Use `--shadow-sm` / `-md` only.
- **Uppercase headings** and **0 radius** are both inherited schema defaults, not chosen settings. They are live, so match them — but know they were never a decision if Ramez wants them changed.
- `default_color_scheme` is empty; Prestige falls back to `color_schemes[0]` , so `scheme-1` is the de-facto default.

6. Admin IA — changed by the design

The design merged Products and Add Product into **one screen**. Build to this, not to the earlier brief.

- `/admin/products` **is the primary and default screen**. There is no dashboard and no KPI summary — a tool used by one person for one task does not need a summary of itself.
- **Add product is a 480px right drawer** over the list. The list stays mounted behind it, so cancelling costs nothing and scroll position is preserved.
- **The fetch result is a modal over the drawer**, not a second pane.
- **Row editing is inline**. Enter commits, Esc reverts, Tab moves to the next cell. This needs a per-field `PATCH` that is safe to call rapidly — debounce on the client, and make the endpoint idempotent.
- Nav destinations: `Products · Collections · Customers`.

Mobile admin is designed and required

Earlier versions of this brief said no phone layout was needed. That is no longer true — `06 · States` and the responsive intent table specify it:

Breakpoint	Admin behaviour
1440	220px nav, full table, inline editing, 480px drawer
810	Table drops Source and Added columns first — ID, Title and Status are what Ramez scans. Drawer keeps its width; the list narrows behind it.
390	Nav → bottom tab bar. Drawer → bottom sheet. Fetch result → full-screen modal. Inline editing is dropped entirely — edits open the sheet instead.

Tablet 810 frames are **specified but not drawn**. Build from the intent table.

7. Customer preview — behaviour the design specifies

Prices are shown

Answered by the design: yes, and the same for every customer. Stored in `Variant Price`

against a single base currency (see §8).

Personalisation

The gate greets by name (`Welcome, Hala.`) and the header carries the name. Comes from `Customers.display_name` . It is not a security boundary — it is on the page before the buyer proves anything beyond the shared password.

Reactions — asymmetric by design

- **Card (grid): like only.** An X beside a heart on a grid invites a verdict on every rug; the card asks one question.
- **Detail page: like and dislike both**, where the buyer has actually looked.
- Record `source` on the event so this stays visible in the data.

Accessibility carried from the components: real focusable buttons with text labels (`Like this rug / Not for me`), never icon-only, pressed state mapped to `aria-pressed` . **The buffered pending state is deliberately not announced** — announcing every buffered write would be intolerable. Visual circles are 32px; the mobile hit area is enlarged to 44px without changing the visual.

Collections carry buyer-facing descriptions

`Collections.description` is now shown to buyers, clamped to one line with a **See more** that expands in place. The field must reach the catalog response.

Footer

Carries `hello@serioludere.com` and the line *“Prices are indicative and convert at an approximate rate.”* There is **no inquiry action** — likes plus the contact address are the reply path.

Spec block

Displays Size, Material, Method, Origin, Pile, Age. **shape is stored but not displayed** — keep the column, do not surface it. Size renders as `240 · 170 cm` (width · length).

8. Units and currency — new backend work

The design added two controls the earlier brief did not cover. The handoff is explicit that rates, refresh cadence and rounding are **a data decision, not a design one** — so they land

here.

Unit toggle — fully specified, no decisions left

- `cm ↔ ft/in`, session-scoped, persists across pages
- **CM is canonical in storage.** Imperial is always derived, never stored.
- `1 cm = 0.3937 in`. Feet-and-inches rounds to the nearest inch; centimetres to the nearest whole cm. Rugs are not sold to the millimetre and false precision reads as a spec sheet.
- Switching reconverts every dimension on the surface at once — cards, detail spec block, filter labels
- Instant, no animation. At 390 the imperial segment abbreviates to `ft`.

Currency picker — decisions required

Supports **USD · EUR · GBP · CAD · MXN · AED · SAR**, session-scoped, reconverts every price at once.

Four things the design deliberately left open:

1. **Base currency.** Pick one, store all prices in it, derive the rest. Recommend USD, since the design shows `$4,850`. Add a store-level constant, not a per-row currency column.
2. **Rate source.** A free daily-rates API (exchangerate.host, Frankfurter) is sufficient — this is an indicative catalog, not a checkout.
3. **Refresh cadence.** Daily is right. Cache the rate table in memory with a long TTL and a hardcoded fallback table, so a rates outage degrades to slightly-stale numbers rather than a page that cannot render prices.
4. **Rounding.** Round converted prices to whole units. Do not show cents on a converted price — it implies a precision the disclaimer explicitly disclaims.

The footer disclaimer is the design's half of this contract. Do not remove it, and do not present a converted price anywhere that reads as a quote.

AED and SAR are in the list because Gulf buyers are expected. No RTL layout has been drawn. The components are auto-layout and would mirror, but that is an assertion, not a tested pass. Flag it before a Gulf buyer gets a link.

9. Products tab

One row per product. Columns follow **Shopify's product CSV import format** so the tab imports without remapping.

```
Product ID          ← manually assigned by Ramez; primary key; unique; rendered i
Handle, Title, Body (HTML), Vendor, Product Category, Type, Tags, Published,
Option1 Name, Option1 Value, Variant SKU, Variant Grams, Variant Inventory Qty,
Variant Inventory Policy, Variant Price, Variant Compare At Price,
Variant Requires Shipping, Variant Taxable, Image Src, Image Alt Text,
SEO Title, SEO Description, Status,
```

```
Width CM, Length CM, Size Label, Size Band, Material, Method, Origin, Age, Pile,
```

```
Collection, Source URL, Source Site, Drive Folder ID, Drive Folder URL,
Scraped At, Commit Status, Internal Notes
```

`Image Src` holds **the primary image only**. All other images live in Drive. A CSV export from this tab carries the primary image only.

`Collection` stores the collection **name** as a plain string, denormalised so the tab stays importable standalone. The `Collections` tab holds the canonical list.

Other tabs:

```
Collections:  id, name, slug, description, created_at          ← description is buy
Customers:    slug, display_name, password_hash, note, created_at, active
Reactions:    event_id, customer_slug, product_id, reaction, source, created_at
Visits:       event_id, customer_slug, occurred_at, user_agent, referrer
```

Reads 300–800ms — cache, 60s TTL. Quota 60 write/min/user, ~300 read/min/project. Product updates need a row index, so read-then-write and address rows by `Product ID`, never position.

10. Auth

Two entirely separate systems. The admin password has no relationship to any customer password.

Admin

One password, no username, no accounts, no recovery. `ADMIN_PASSWORD` env var. Signed HTTP-only, `SameSite=Lax`, `Secure` cookie, 7-day expiry, signed with `AUTH_SECRET`. Constant-time comparison.

Ramez wants to change it “from the code” — an env var is that, minus the risk of committing a password to git. A password in git history cannot be removed.

Customer

One **generated** password per customer, unrelated to the admin password and to every other customer.

- Generated server-side, never typed by Ramez
- Format: three lowercase words + two digits (`amber-loom-serai-47`) — readable over the phone, pasteable into WhatsApp, no ambiguous characters
- Stored as an **argon2 or bcrypt hash**. Never plaintext — the sheet is visible to anyone with Drive access.
- Shown **once** at generation via the reveal-once credential panel. If lost, regenerate: `POST /api/customers/[slug]/regenerate` returns new plaintext once and stores a new hash. The old one dies.

Decision needed. Hashing means a forgotten password can only be regenerated, not looked up. Storing it reversibly is more convenient and weaker — one leak of the spreadsheet opens every customer link. Recommendation: hash + regenerate.

Both

- Cookies scoped per realm; a customer cookie must not unlock another slug or `/admin`
- Rate limit login POSTs: 5 per IP per 15 min, in memory
- Reserved slugs blocked at creation: `admin` , `api` , `login` , `logout` , `_astro` , `assets` , `favicon.ico`
- Middleware default-denies

State plainly: a link plus a password is a gate, not access control. A forwarded link is indistinguishable from the intended buyer, and the analytics inherit that.

11. Scraping

Server-side only — CORS blocks browser `fetch` to third-party sites, with no client-side workaround. **One URL per request. No bulk mode.**

Two-step contract — the core rule

```
POST /api/scrape    → fetch + normalise + return JSON.  WRITES NOTHING.
POST /api/products  → commit the reviewed payload to Drive + Sheet.
```

Nothing reaches Sheets or Drive until Ramez presses Save. The commit endpoint takes the **edited** payload — it must not re-scrape.

Response ordering matters to the UI

The design reveals **the photo first, then the fields** (300ms photo, then fields at 200ms with a 40ms stagger). The photo is how Ramez tells whether the scrape found the right rug, so a wrong result is cancelled before a single field is read. Either stream the response or return the primary image URL in a first chunk — do not make the client wait for full normalisation before it can show an image.

Fetch progress is indeterminate. A scrape has no honest percentage; the stage label carries the real information (`fetching page` → `parsing` → `resolving images`). Do not fake a progress bar. Commit progress *is* stepped and counted, because there the counts are real.

Adapters

```
src/lib/scrapers/
  index.ts  shopify.ts  jsonld.ts  ecarpetgallery.ts  karavan.ts  generic.ts  nor
```

Extraction ladder — first good result wins

1. **Shopify JSON endpoint.** Try `{product-url}.js` and `.json`. Many rug retailers run Shopify; a hit gives title, `body_html`, vendor, tags, full image list, variants and prices structurally, with zero HTML parsing. Check first for every URL.
2. **JSON-LD** — `<script type="application/ld+json">` → `Product`
3. **OpenGraph / microdata**
4. **Per-source CSS selectors** — last resort, most fragile

`cheerio` for HTML. **No Playwright/Puppeteer in v1.**

Normalised shape

```
type ScrapedProduct = {
  title: string | null
  descriptionHtml: string | null
  price: { amount: number; currency: string } | null
```

```

images: string[] // ordered; [0] is primary
dimensions: { widthCm: number; lengthCm: number } | null
attributes: Record<string, string> // material, method, origin, pile, age,
sourceUrl: string
sourceSite: string
fieldStatus: Record<string, 'found' | 'inferred' | 'missing'>
}

```

`fieldStatus` drives the missing-field flagging in the fetch modal. Populate it honestly.

Sizes appear as 5'3" x 7'6" , 160x230 , 160 x 230 cm . **Normalise to CM.** Round. Size

band from area: <0.75 m² XS · ≤2.5 S · 2.5-5 M · 5-10 L · >10 XL . Size Label

renders as 240 · 170 cm .

Robustness

15s timeout, one retry with backoff. Realistic `User-Agent` , respect `robots.txt` , ≥2s between requests to the same host. Never throw a bare 500 — return partial data with `fieldStatus` set. Distinguish `blocked` / `not_found` / `parse_failed` / `timeout` .

12. Google Drive image pipeline

```

/Serio Ludere - Preview Catalog/
  /{Product ID} - {Product Name}/
    01-primary.jpg           ← first image, at this level
  /All Images/
    01.jpg 02.jpg ... 12.jpg ← every image, primary included

```

The primary is duplicated deliberately: the folder-level copy is what the sheet references and both previews render.

Commit sequence — order matters

1. Validate Product ID is unique → reject before any side effect
2. Write sheet row, Commit Status=pending
3. Create Drive folders
4. Download + upload images sequentially, streaming progress to the client
5. Update row: Image Src, Drive Folder ID/URL, Commit Status=complete

Row first — a failure is then visible and retryable rather than a silent half-save with orphaned folders. Rows stuck at `pending` surface in the products list with the failed-commit row

treatment and a retry action.

Drive auth — resolve before writing upload code

A service account is the obvious choice and the wrong one. Files it creates in an ordinary My Drive folder are **owned by the service account**, which has no storage quota — uploads fail, or land where Ramez cannot delete them. Two viable paths:

- **Google Workspace + a Shared Drive** — files owned by the drive; service account works cleanly
- **OAuth refresh token on Ramez’s own account** — one consent flow, token in an env var. Files owned by him, and the same credential covers Sheets. The pragmatic route on free Gmail.

Serving

Drive is not a CDN — `uc?id=` links are rate-limited, sometimes hit an interstitial, and thumbnail links expire. Serve through `/api/image/[fileId]` with aggressive cache headers.

Limits

20 images per product, 10MB each. Uploads are sequential: a 12-image product takes 10–20s. **Check the platform function timeout** — Vercel Hobby is 10s. If tight, batch uploads across several client-initiated requests.

13. Motion — implement from 07 · Handoff

Per element, never a global default. Transform and opacity only; nothing animates layout.

Element	Duration	Easing
Product card image hover	600ms, 130% zoom	ease-out — extracted, the storefront’s real values
Card grid entry	400ms, 60ms stagger, capped at 12 cards	ease-out
Product detail hero	500ms, opacity only, no scale	ease-out
Like/dislike tap	120ms, scale 1 → 0.92 → 1	ease-out
Pending dot	none — deliberately static	—

Drawer in / out	240ms / 180ms	ease-out / ease-in
Bottom sheet	280ms / 200ms	ease-out / ease-in
Fetch photo reveal	300ms	ease-out
Fetch fields reveal	200ms, 40ms stagger	ease-out
Currency menu	150ms / 100ms	ease-out / ease-in
Unit toggle	none — deliberately instant	—
Toast in / out	200ms / 150ms	ease-out / ease-in
Admin, everything else	150ms	ease-out

Two inherited behaviours: Prestige's **button hover wipe** is already gated behind `(pointer: fine)` and `prefers-reduced-motion` — inherit it, do not reimplement. Prestige emits `@view-transition { navigation: auto }` — **the preview inherits it; the admin must not**, since a view transition on a data table reads as lag.

`prefers-reduced-motion` collapses everything to instant. Prestige already honours this; match its posture rather than inventing one.

14. Routes

```

GET  /admin/login
POST /api/auth/admin
GET  /admin/products           → default screen; drawer + modal are
GET  /admin/collections | /admin/customers
GET  /admin/customers/[slug]   → visits + liked/disliked

POST /api/scrape               → fetch + normalise, photo-first res
POST /api/products             → commit: Drive + Sheet
PATCH /api/products/[id]      → inline row edit; idempotent, debou
DELETE /api/products/[id]
POST /api/products/[id]/retry
GET|POST|DELETE /api/collections
GET|POST|PATCH /api/customers → POST returns generated password on

```

```

POST /api/customers/[slug]/regenerate
GET /api/customers/[slug]/reactions → derived current state
POST /api/admin/compact-reactions
GET /api/image/[fileId]
GET /api/rates → cached daily FX table with static
GET /api/export/shopify-csv

POST /api/auth/customer/[slug]
POST /api/reactions → accepts a batch; each item carries
GET /[slug] → gate or catalog
GET /[slug]/[productId]

```

`/[slug]` is a catch-all and must be the **last** route — verify it does not shadow `/admin`.

15. Honest assessment

1. Sheets works here only with the append-only design in §3. Any mutable read-modify-write on reactions loses events silently under concurrency — no error, no log, just a like that never appears. If §3 is skipped as an optimisation, the feature is quietly broken.

2. The ceiling is real but not near. A few hundred products and a handful of buyers is comfortable. You have hit the wall when the reaction log passes ~20,000 rows, admin lists take over two seconds, or compaction is needed more than monthly. Keep sheet access behind a repository interface so the move to SQLite/Turso is a day's work.

3. Scraping is permanently brittle. Every source breaks its parser eventually, without warning. Maintenance cost, not a one-time bug. The Shopify-JSON-first ladder and the mandatory review step are what make it survivable.

4. Drive ownership and Drive-as-CDN bite late. Resolve §12's auth model before writing upload code.

5. Copyright. This copies supplier photos into your own storage and serves them to customers — a larger exposure than hotlinking. A password gate reduces it, does not remove it. Recommendation: treat scraped copy and photos as internal sourcing data, and rewrite descriptions and reshoot or license images before a buyer sees them.

6. Analytics attribute to a link, not a person. A forwarded link is indistinguishable from the intended buyer, and one link cannot separate two devices.

7. Currency conversion is a promise. The footer says prices are indicative. Keep it there, keep rounding coarse, and never render a converted price in a context that reads as a quote.

16. Not in the Figma file — build from spec, not from frames

Reported in the handoff rather than left to discover:

- **Tablet 810 frames** — intent specified (§6), frames not drawn
 - **Prototype wiring** — the flows on `03 · Flows` carry the full trigger → behaviour → goes-to map, so transitions are specified but not wired
 - **Cross-customer roll-up** on the Customers index — specified in Flow 6, not drawn
 - **Disliked grid** on customer detail — specified; reuses the Liked grid with a muted plate
 - **RTL / Arabic mirror** — not drawn, despite AED and SAR in the picker
 - **Data Table as one organism** — Table Row, Inline Row and Filter Bar exist and compose the screen; they were not extracted
 - **Currency rates, refresh and rounding** — deliberately left to this brief (§8)
-

17. Sequence

1. Astro SSR scaffold, adapter, subdomain, env
2. **Resolve Drive auth model (§12)** — blocking
3. Generate CSS custom properties from the Figma variables, three modes (§5)
4. Middleware + admin auth + login
5. Sheets repo layer + Collections CRUD (*smallest end-to-end slice*)
6. `/api/scrape` with the ladder, photo-first response, tested against 10 real URLs per source
7. Drive pipeline + commit sequence with pending/complete
8. Products screen: list, drawer, fetch modal, inline editing, retry
9. Customers CRUD + password generation + link copy
10. `/[slug]` gate + catalog + product detail, with unit toggle and currency picker
11. Reactions (append-only, batched, `source`-aware) + visit tracking + admin analytics + compaction

12. Mobile admin: tab bar, bottom sheet, full-screen fetch modal

13. Shopify CSV export

Ship 1–8 and use it before building 9–13. The scraper’s real failure modes only surface against real URLs.

18. Environment

```
ADMIN_PASSWORD
AUTH_SECRET
GOOGLE_SHEET_ID
GOOGLE_DRIVE_ROOT_FOLDER_ID
BASE_CURRENCY           # recommend USD
FX_API_URL              # daily rates, with a hardcoded fallback table in code
PUBLIC_SITE_URL

# one of:
GOOGLE_SERVICE_ACCOUNT_EMAIL + GOOGLE_PRIVATE_KEY      # Shared Drive route
GOOGLE_OAUTH_CLIENT_ID + SECRET + REFRESH_TOKEN       # personal-account route
```

Enable the Sheets API and the Drive API. Share the spreadsheet and the Drive root folder with the chosen identity as Editor. Store private keys with escaped newlines.